Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307P - Compiler Design Lab

Project Report: Review 1

Project Title: Current Limitations in Compiler Design

Team Member(s): Max Three members

S. No	Register No	Name	Contributions
1.	24BCE0484	Divyesh B	Limitation 1 (Front-end): Cascading and cryptic syntax errors. Problem identification, objectives, literature survey (S.No 1-7), novelty and presentation for Limitation 1; design and implementation of a MiniC recursive-descent parser with grammar-guided error recovery and repair suggestions.
2.	24BDS0253 24BDS0279	Advik K Nama Adarsh Kumar Jha	Limitation 2 (Middle-end): The phase-ordering problem in optimization passes. Problem identification, objectives, literature survey (S.No 8-12), research gap and presentation for Limitation 2; design and implementation of the TAC optimizer core (constant propagation and constant folding passes) and the permutation-based evaluation harness.

Problem Identification & Objectives - 2M

This project studies current limitations in compiler design. Each team member takes up one limitation independently, and the sections below are organised limitation by limitation.

Problem Statement

Limitation 1: Cascading and Cryptic Syntax Errors (Front-end, Syntax Analysis) - Divyesh B, 24BCE0484

Syntax analysis is where most programmer mistakes are first detected, yet the way parsers report them has changed little in practice. Most hand-written and textbook parsers use panic-mode recovery: on an error, they discard input tokens until a synchronising token such as ";" or "}" appears. Because the parser resumes without knowing what actually went wrong, it often restarts in an inconsistent state and reports further "errors" that are only consequences of the first one. A single slip, such as a missing ")" or a misspelt keyword, can therefore produce a chain of misleading messages. The messages themselves are phrased in parser terms (for example, "syntax error near {"), frequently point at the wrong line, and rarely say how to fix the code.

Illustrative example (naive panic-mode parser):

```
1  while (i < 10 {
2      sum = sum + i;
3      i = i + 1;
4  }

line 1: syntax error near '{'
line 2: unexpected 'sum'
line 3: unexpected 'i'
line 4: unexpected '}'
```

One real mistake (the missing ")" on line 1) produces four reported errors.

Well-studied repair algorithms exist, but they are designed around LR/LL parse tables or parsing

expression grammars. Recursive-descent parsers, which are widely used in practice and in teaching, are usually left with ad-hoc skip-based recovery. Newer learned approaches (deep learning and large language models) need training data or an external service, are non-deterministic, and work outside the compiler. The need addressed by this project is a lightweight, deterministic recovery layer for recursive-descent parsers that reports each real error once, at the right place, with a concrete fix, and whose improvement can be measured.

Limitation 2: The Phase-Ordering Problem in Compiler Optimization (Middle-end, Optimization) - Advik K Name, 24BDS0253 & Adarsh Kumar Jha, 24BDS0279

Compilers apply optimization passes such as constant propagation, constant folding, dead-code elimination and common subexpression elimination (CSE) one after another, but these passes are not independent of one another. Applying one pass can create new opportunities for another: constant folding can replace a variable with a literal value, which can turn a branch or assignment into dead code that dead-code elimination can then remove; removing that dead code can in turn expose duplicate expressions that CSE is able to merge. Because each pass changes what the next pass sees, the order in which the passes are run can change the size and the runtime behaviour of the generated code, so running the same set of passes as A -> B instead of B -> A can produce different results. Deciding this order, known in compiler research as the phase-ordering problem, has been studied for decades and is still considered largely unresolved [9].

The difficulty is that the number of possible pass orderings grows combinatorially with the number of passes and the length of the sequence, so for a compiler with even a modest set of passes, trying every ordering exhaustively for every function quickly becomes impractical [11]. Production compilers sidestep this by shipping one fixed, hand-tuned order (such as -O1, -O2 or -O3) that is applied to every program, even though no single fixed order is known to be best for all programs and code segments [8]. Prior work has approached the problem through exhaustive or near-exhaustive search on constrained pass sets [9], by identifying and removing accidental (false) dependencies between passes to shrink the search space [10], and by using machine learning to predict good sequences or sub-sequences of passes without exploring the whole space [11], [12]. This project studies the phase-ordering problem directly and empirically for a small, teachable set of four classic passes: it implements a 3-Address Code (TAC) optimizer in Python with constant propagation, constant folding, dead-code elimination and CSE, then evaluates every permutation of this pass sequence on representative TAC programs and compares the resulting instruction counts, to make concrete, for a small controlled pass set, how much the order of optimization passes matters.

Objectives (Minimum 3)

Limitation 1: Cascading and Cryptic Syntax Errors

- **Build the baseline:** implement a lexer and recursive-descent parser for MiniC, a small C-like language (declarations, assignment, if-else, while, print, arithmetic and relational expressions), with standard panic-mode recovery as the reference. It must parse all valid programs in the test suite correctly.
- **Suppress cascading errors:** add FOLLOW-set synchronisation and an error-suppression window (no new error reported until k tokens, e.g. 3, parse cleanly). Target a Cascade Ratio (errors reported / actual errors) of at most 1.2, compared with the baseline.
- **Suggest concrete repairs:** at each error, try single-token insert, delete and replace candidates, re-parse each, and report the one that parses furthest. Target the correct repair for at least 70% of single-token errors.
- **Handle misspellings:** give "did you mean ...?" suggestions for misspelt keywords and identifiers using Levenshtein edit distance (distance at most 2). Target the correct suggestion for at least 90% of injected misspellings.
- **Evaluate:** build a suite of at least 30 erroneous programs by injecting known errors into correct programs, and compare baseline and improved parsers on Cascade Ratio, exact line:column location accuracy (target at least 90%) and repair accuracy.

Limitation 2: The Phase-Ordering Problem in Compiler Optimization

- **Build the baseline TAC optimizer:** implement a lexer/IR loader and four classic optimization passes (constant propagation, constant folding, dead-code elimination, common subexpression elimination) operating on 3-Address Code, each individually verified to preserve program semantics on a suite of small TAC programs.
- **Enumerate all pass-sequence permutations:** for the four passes, run every possible ordering (iterated to a fixed point or a fixed number of rounds) on every test program and record the resulting instruction count and code size for each ordering.
- **Quantify the phase-ordering effect:** compute, for each test program, the spread between the best and worst instruction counts across all orderings, and identify the recurring pass-interaction patterns (e.g. constant folding enabling dead-code elimination, dead-code elimination enabling CSE) that explain why certain orderings dominate.
- **Compare against fixed heuristics:** benchmark the best order found by exhaustive permutation search against one or two common fixed orders (e.g. a typical textbook pipeline order) and report the percentage of test programs on which permutation search finds a strictly better order.
- **Evaluate:** build a suite of at least 15-20 representative TAC programs (straight-line code, loops, branches) with deliberately placed optimization opportunities, and report instruction-count and code-size results across all orderings for each program, targeting a clear demonstration that no single fixed order is optimal for every program in the suite.

Literature Survey - 2M

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
Limitation 1: Cascading and Cryptic Syntax Errors (Divyesh B, 24BCE0484)				
1	A minimum distance error-correcting parser for context-free languages / A. V. Aho, T. G. Peterson / 1972	Global (minimum-distance) correction: finds the fewest symbol insertions, deletions and replacements that make the whole input valid for any context-free grammar.	Guarantees a least-cost correction for every erroneous input.	Cubic time in input length; impractical for real compilers and gives one global answer with no local explanation.
2	A practical method for LR and LL syntactic error diagnosis and recovery / M. G. Burke, G. A. Fisher / 1987	Three stages: primary recovery tries single-token insert, delete or replace; scope recovery inserts missing closers; secondary recovery falls back to panic mode. Uses deferred parsing.	High-quality local repairs for both LR and LL parsers.	Built around parse tables and a deferred parse stack; complex to adapt to hand-written recursive-descent parsers.
3	DeepFix: Fixing common C language errors by deep learning / R. Gupta, S. Pal, A. Kanade, S. Shevade / 2017	Sequence-to-sequence neural network with attention, trained on student C programs; fixes errors iteratively.	Fixes a substantial share of erroneous student programs automatically.	Needs large training data; non-deterministic; works outside the compiler, which still reports its original cascade.
4	Using large language models to enhance programming error messages / J. Leinonen et al. / 2023	Uses a large language model (Codex) to explain error messages and suggest fixes for novice programs.	Explanations are mostly understandable, but suggested fixes are only sometimes correct.	Non-deterministic, may be wrong with confidence, needs an external API; not part of the compiler.
5	Follow set error recovery / C. Stirling / 1985	Recovery for recursive-descent parsers: each procedure receives the set of tokens that may follow it and skips input until one of them appears.	Simple, systematic recovery that fits hand-written recursive-descent parsers.	Skip-only: no repair is proposed to the programmer; spurious follow-on errors are still possible and the effect is not quantified.

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
6	Don't Panic! Better, fewer, syntax errors for LR parsers / L. Diekmann, L. Tratt / 2020	CPCT+ algorithm finds the complete set of minimum-cost repair sequences at each error location; implemented in the Rust grmtools library and evaluated on real erroneous Java programs.	Repairs found quickly in the vast majority of cases, with far fewer cascading errors than panic mode.	LR-only; not applicable to hand-written recursive-descent parsers.
7	Do developers read compiler error messages? / T. Barik et al. / 2017	Eye-tracking study of developers resolving compiler errors.	Developers do read error messages, and reading them is about as difficult as reading source code.	Characterises the problem; proposes no recovery technique.
Limitation 2: The Phase-Ordering Problem (Advik K Name, 24BDS0253 & Adarsh Kumar Jha, 24BDS0279)				
8	Adaptive optimizing compilers for the 21st century / K. D. Cooper, D. Subramanian, L. Torczon / 2002	Iterative/adaptive compilation: hill-climbing and genetic-algorithm search over sequences of optimization passes, using measured performance as feedback to guide the search.	Search-based methods can find pass sequences that beat a single fixed compilation order for individual programs.	No single fixed order is best for all programs, but search is expensive; motivates studying the phase-ordering effect directly rather than assuming one true order.
9	Practical exhaustive optimization phase order exploration and evaluation / P. A. Kulkarni, D. B. Whalley, G. S. Tyson, J. W. Davidson / 2009	Prunes and exhaustively enumerates the phase order search space for a compiler backend using properties of the passes, then measures performance for every remaining ordering.	Exhaustive exploration is feasible for realistic pass sets once the space is pruned, and reveals that phase order has a substantial effect on code quality.	Pruning techniques are tuned to a specific production backend and large pass sets; leaves open a small, easily-reproduced demonstration of the same effect for a handful of classic passes.
10	Eliminating false phase interactions to reduce optimization phase order search space / M. R. Jantz, P. A. Kulkarni / 2010	Analyses why optimization phases appear to interact, identifies false (register-dependence-driven) interactions, and adds cleanup phases to remove them before searching.	Removing false interactions substantially shrinks the phase order search space and can also improve the code the compiler generates.	Focuses on removing spurious interactions rather than characterising or teaching the genuine interactions (such as constant folding enabling dead-code elimination) that this project studies directly.
11	MiCOMP: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning / A. H. Ashouri, A. Bignoli, G. Palermo, C. Silvano, S. Kulkarni, J. Cavazos / 2017	Clusters LLVM's O3 passes into sub-sequences and uses machine learning to predict the speedup of a full ordering of clusters, avoiding a per-pass search.	Reaches most of the available speedup from exhaustive search while exploring only a tiny fraction of the full search space.	Needs a large pass set, training data and a production compiler; gives a predicted best order rather than the exact, exhaustively verified comparison this project performs on a small pass set.
12	A survey on compiler autotuning using machine learning / A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, C. Silvano / 2018	Surveys machine-learning approaches to compiler autotuning, separating the optimization-selection problem from the harder phase-ordering problem and comparing search and prediction strategies.	Phase ordering is consistently reported as harder than selection and remains largely open; most learning-based methods approximate rather than exhaustively solve it.	Confirms that small-scale, fully-enumerable studies of phase ordering are uncommon in the surveyed literature, which this project's exhaustive-permutation approach on a compact pass set addresses.
13	AutoPhase: Compiler phase-ordering for HLS with deep reinforcement learning / Q. Huang, A. Haj-Ali, W. Moses, J. Xiang, I. Stoica, K. Asanović, J. Wawrzynek / 2019	Formulates pass selection as a reinforcement learning problem: a deep RL agent observes LLVM IR features and learns a policy that chooses the next optimization pass to apply, evaluated on LLVM-based high-level synthesis programs.	The learned policy finds pass sequences that improve circuit performance by up to 16% over -O3 and runs one to two orders of magnitude faster than prior phase-ordering search algorithms.	Needs a trained RL policy and a circuit-performance reward signal tied to HLS synthesis; does not exhaustively verify results or expose which specific pass-interactions drive the improvement.

S.No	Title / Author / Year	Methodology	Findings	Gap Identified
14	Iterative compilation in a non-linear optimisation space / F. Bodin, T. Kisuki, P. M. W. Knijnenburg, M. F. P. O'Boyle, E. Rohou / 1998	Case study applying profile-feedback-guided iterative search directly over a non-linear space of program transformation parameters, using measured execution time rather than a static cost model to guide the search.	A simple iterative search algorithm reaches within 0.3% of the best achievable execution time while visiting under 1% of the full search space.	Targets numerical transformation parameters (e.g. tiling and unrolling) rather than the ordering of independent optimization passes, and does not examine pass-enables-pass interactions such as constant folding enabling dead-code elimination.
15	Stochastic superoptimization / E. Schkufza, R. Sharma, A. Aiken / 2013	Formulates optimization of loop-free binary code as a stochastic search over the space of candidate programs using Markov Chain Monte Carlo sampling, combining correctness and performance in a single cost function.	The STOKE prototype produces x86 binaries that match or outperform gcc -O3 output, starting from unoptimized code, and in some cases rivals expert hand-written assembly.	Searches the space of output programs directly rather than the space of pass orderings, so it sidesteps rather than explains the phase-ordering problem, and is restricted to loop-free code with an expensive per-function sampling loop.
16	CompilerGym: Robust, performant compiler optimization environments for AI research / C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, H. Leather / 2022	Packages real LLVM phase-ordering, GCC flag-selection and CUDA loop-generation problems as standardised reinforcement-learning environments with large program datasets, enabling large-scale evaluation of autotuning and RL techniques.	The LLVM phase-ordering environment alone exposes a search space far larger than the game of Go and runs up to 27x faster than prior infrastructure, letting many RL and autotuning techniques be benchmarked on production-scale phase ordering.	Provides infrastructure and a benchmark for large-scale learned search over production LLVM pass sets, not a small, fully-enumerable, pedagogically transparent demonstration of why pass order matters — the gap this project addresses directly.

Research Gap and Motivation

Limitation 1: Cascading and Cryptic Syntax Errors

The surveyed work falls into three strands. Error-correcting and repairing parsers [1], [2], [6] can remove cascading errors and propose fixes, but they are either too slow for practical use [1] or depend on LR/LL parse tables [2], [6]. The one technique aimed at recursive-descent parsers [5] only skips input and never proposes a repair. Barik et al. [7] show that programmers do read error messages and find them as hard to read as source code, so misleading cascades and wrong locations have a real cost. Learned approaches [3], [4] produce useful fixes in many cases, but they need training data or an external service, are non-deterministic, and work outside the compiler. There is therefore no lightweight, deterministic technique that fits inside a hand-written recursive-descent parser, both stops cascades and suggests a concrete fix, and is backed by a measured result. This gap motivates Limitation 1 of the project.

Limitation 2: The Phase-Ordering Problem in Compiler Optimization

The surveyed work on phase ordering falls into three groups. Exhaustive and near-exhaustive search methods [9] give ground-truth answers but do not scale past a modest number of passes because the search space grows combinatorially. Search-space-reduction techniques [8], [10] cut this cost by identifying false dependencies between passes or by using adaptive search over reordering and deletion, but they still target large, industrial pass sets and require substantial tooling to reproduce. Machine-learning approaches [11], [12] predict good sequences or sub-sequences without exhaustive search, but they need training data, a compiler with dozens of passes, and are aimed at production settings rather than at showing, clearly and reproducibly, why phase ordering matters in the first place. More recent search paradigms extend this picture without closing the gap: deep reinforcement learning [13] and standardised RL environments [16] scale phase-ordering search to production compilers but still require large training infrastructure, while iterative compilation [14] and stochastic superoptimization [15] search adjacent spaces (transformation parameters and instruction sequences,

respectively) rather than exhaustively enumerating pass orderings. What is missing is a small, fully-enumerable and pedagogically transparent demonstration of the phase-ordering problem: one that implements a handful of well-understood passes from scratch, exhaustively runs every ordering (feasible only because the pass set is small), and directly reports the resulting spread in code quality along with the pass-interaction patterns that cause it. This gap motivates Limitation 2 of the project.

Novelty - 2M

Limitation 1: Cascading and Cryptic Syntax Errors

The novelty lies in adapting and combining existing ideas for hand-written recursive-descent parsers and measuring the result. It does not claim a new repair algorithm.

- **Table-free repair in a recursive-descent parser:** Burke-Fisher and CPCT+ style local repairs [2], [6] rely on LR/LL parse tables. This project finds repairs by re-parsing candidate token edits directly in a hand-written recursive-descent parser, so no tables or parser generator are needed.
- **Repair first, synchronise second:** follow-set recovery for recursive descent [5] only skips input. Here a repair is attempted first; FOLLOW-set synchronisation plus an error-suppression window is used only as a fallback, which targets cascades directly.
- **Actionable messages generated from the recovery itself:** rather than generic parser messages, each message is produced from the chosen repair: exact line:column, the fix ("insert ')' before '{'"), and "did you mean ...?" for misspellings. This addresses the reading difficulty reported in [7].
- **Deterministic and offline:** unlike DeepFix [3] and LLM-based tools [4], the approach needs no training data, GPU or network access, and always gives the same output for the same input.
- **Measured, not just claimed:** as in the evaluation of [6] but for a recursive-descent parser, a mutation-generated test suite and three simple metrics (Cascade Ratio, location accuracy and repair accuracy) compare the baseline and improved parser quantitatively.

Limitation 2: The Phase-Ordering Problem in Compiler Optimization

- **Exhaustive, not sampled, search over a small pass set:** unlike [9]-[12] and the deep-RL frameworks in [13], [16], which reduce or approximate the search space because their pass sets are large, this project's four-pass set is small enough that every permutation can be run and compared directly, giving an exact rather than an estimated picture of the phase-ordering effect.
- **Built from scratch on a custom TAC IR:** rather than instrumenting an existing production compiler as in [8], [10], [11], each pass is implemented directly over a simple 3-Address Code representation, making the pass-interaction effects easy to trace and explain.
- **Explicit measurement of pass-interaction patterns:** beyond reporting which order is best, the project identifies and reports the specific pass-enables-pass relationships (e.g. constant folding enabling dead-code elimination, which in turn enables CSE) responsible for the improvement, connecting the empirical result back to its underlying cause.
- **Reproducible, deterministic comparison:** since the pass set and program suite are fixed and the search is exhaustive, results are exactly reproducible, unlike the sampled or learned approaches in [11], [12].

References

[1] A. V. Aho and T. G. Peterson, "A minimum distance error-correcting parser for context-free languages," SIAM Journal on Computing, vol. 1, no. 4, pp. 305-312, 1972.

[2] M. G. Burke and G. A. Fisher, "A practical method for LR and LL syntactic error diagnosis and recovery," ACM Transactions on Programming Languages and Systems, vol. 9, no. 2, pp. 164-197, 1987.

[3] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in Proc. AAAI, 2017.

[4] J. Leinonen et al., "Using large language models to enhance programming error messages," in Proc. ACM

SIGCSE, 2023, pp. 563-569.

[5] C. Stirling, "Follow set error recovery," Software: Practice and Experience, vol. 15, 1985.

[6] L. Diekmann and L. Tratt, "Don't Panic! Better, fewer, syntax errors for LR parsers," in Proc. ECOOP, LIPIcs vol. 166, 2020.

[7] T. Barik, J. Smith, K. Lubick, E. Holmes, J. Feng, E. Murphy-Hill, and C. Parnin, "Do developers read compiler error messages?" in Proc. IEEE/ACM ICSE, 2017, pp. 575-585.

[8] K. D. Cooper, D. Subramanian, and L. Torczon, "Adaptive optimizing compilers for the 21st century," The Journal of Supercomputing, vol. 23, no. 1, pp. 7-22, 2002.

[9] P. A. Kulkarni, D. B. Whalley, G. S. Tyson, and J. W. Davidson, "Practical exhaustive optimization phase order exploration and evaluation," ACM Transactions on Architecture and Code Optimization, vol. 6, no. 1, Article 1, 2009.

[10] M. R. Jantz and P. A. Kulkarni, "Eliminating false phase interactions to reduce optimization phase order search space," in Proc. Int'l Conf. on Compilers, Architectures and Synthesis for Embedded Systems (CASES), 2010, pp. 87-96.

[11] A. H. Ashouri, A. Bignoli, G. Palermo, C. Silvano, S. Kulkarni, and J. Cavazos, "MiCOMP: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning," ACM Transactions on Architecture and Code Optimization, vol. 14, no. 3, Article 29, 2017.

[12] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," ACM Computing Surveys, vol. 51, no. 5, Article 96, 2018.

[13] Q. Huang, A. Haj-Ali, W. Moses, J. Xiang, I. Stoica, K. Asanović, and J. Wawrzynek, "AutoPhase: Compiler phase-ordering for HLS with deep reinforcement learning," in Proc. IEEE 27th Int'l Symp. on Field-Programmable Custom Computing Machines (FCCM), 2019, p. 308.

[14] F. Bodin, T. Kisuki, P. M. W. Knijnenburg, M. F. P. O'Boyle, and E. Rohou, "Iterative compilation in a non-linear optimisation space," in Proc. Workshop on Profile and Feedback-Directed Compilation (PFDC), held in conjunction with PACT, 1998.

[15] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic superoptimization," in Proc. ACM Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2013, pp. 305-316.

[16] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, performant compiler optimization environments for AI research," in Proc. IEEE/ACM Int'l Symp. on Code Generation and Optimization (CGO), 2022, pp. 92-105.